

UNIVERSIDAD NACIONAL DE MOQUEGUA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

GUÍA DE TRABAJO PRÁCTICO

**Niveles de Prueba de Software: Unitarias,
de Integración y End-to-End con Docker**

Asignatura: IS-722 Calidad de Software

Semestre Académico: 2026-I

Ciclo: VII Ciclo

Docente Responsable: Mgr. Juan Carlos Valero Gómez

Moquegua, Perú
2026

1. Objetivo de la Práctica

Capacitar a los estudiantes en el diseño, implementación y ejecución de los tres niveles fundamentales de prueba de software (unitarias, de integración y end-to-end) sobre una aplicación real containerizada.

Cada estudiante deberá comprender las diferencias conceptuales entre cada nivel, el costo asociado a su ejecución y el criterio de ingeniería para decidir qué cantidad de pruebas escribir en cada uno, en concordancia con el modelo de la pirámide de pruebas.

Al finalizar la práctica, el estudiante estará en capacidad de:

- Levantar un entorno de pruebas reproducible mediante Docker y Docker Compose.
- Escribir pruebas unitarias empleando dobles de prueba (*mocks*) y el patrón Arrange-Act-Assert.
- Escribir pruebas de integración contra una base de datos relacional real.
- Escribir pruebas end-to-end de caja negra sobre el sistema completo en ejecución.
- Interpretar los reportes de ejecución de Jest y diagnosticar fallos.
- Argumentar, con evidencia empírica de tiempos de ejecución, la distribución recomendada de pruebas según la pirámide.

2. Fundamento Teórico

2.1. Los tres niveles de prueba

Cuadro 1: Comparativa de los Niveles de Prueba de Software

Nivel	Qué verifica	Qué usa	Velocidad
Unitaria	Una unidad de código aislada (una función, una clase).	Dobles de prueba (<i>mocks</i>); no toca red ni disco.	Milisegundos
Integración	Que dos o más componentes trabajen correctamente juntos.	Base de datos real; la aplicación en memoria.	Décimas de segundo
End-to-End (E2E)	El sistema completo desde la perspectiva del usuario.	Servidor HTTP real, base de datos real.	Segundos

2.2. La pirámide de pruebas

La pirámide de pruebas establece que un proyecto sano debe contener muchas pruebas unitarias en la base, menos pruebas de integración en el medio y pocas pruebas end-to-end en la cúspide.

La razón no es dogmática, sino económica: conforme se asciende en la pirámide, la prueba proporciona mayor confianza (verifica el sistema más parecido al real) pero incrementa su costo en tiempo de ejecución, su fragilidad ante cambios y la dificultad de diagnosticar la causa raíz de un fallo.

Principio rector

Una prueba solo detecta aquello que verifica explícitamente. Un conjunto de pruebas end-to-end en verde no significa "no hay defectos"; significa "el flujo que probé funciona". Este principio será demostrado empíricamente en la Actividad 5.

2.3. Inyección de dependencias como habilitador de la testabilidad

La aplicación de esta práctica implementa inyección de dependencias: la capa de servicio recibe el pool de conexiones a PostgreSQL como parámetro en cada función, en lugar de importarlo directamente.

Esta decisión de diseño es la que hace posible que la misma función sea probada con una base de datos falsa (nivel unitario) o con una base de datos real (niveles de integración y E2E) sin modificar una sola línea del código de producción.

3. Descripción del Sistema Bajo Prueba

3.1. Naturaleza de la aplicación

Se trata de una API REST de gestión de tareas (*to-do list*) desarrollada en Node.js con Express, que persiste la información en PostgreSQL empleando SQL directo mediante el driver `pg` (sin ORM).

La API no posee interfaz gráfica: responde peticiones HTTP en formato JSON.

Recursos expuestos

Verbo	Ruta	Descripción
GET	<code>/health</code>	Verifica que el servicio está operativo
GET	<code>/tasks</code>	Lista todas las tareas registradas
GET	<code>/tasks/:id</code>	Recupera una tarea específica
POST	<code>/tasks</code>	Crea una nueva tarea (valida que el título no sea vacío)
PUT	<code>/tasks/:id</code>	Actualiza el título y/o el estado de una tarea
DELETE	<code>/tasks/:id</code>	Elimina una tarea

3.2. Arquitectura por capas

El código se organiza bajo el directorio `src/` de la siguiente manera:

- `src/app.js`: Fábrica de la app Express (recibe el pool inyectado).
- `src/server.js`: Punto de entrada: levanta el servidor HTTP real.
- `src/db.js`: Creación del pool de conexiones a PostgreSQL.
- `src/routes/tasks.routes.js`: Mapa de URLs → funciones del controlador.
- `src/controllers/tasks.controller.js`: Traductor entre HTTP y la lógica de negocio.
- `src/services/tasks.service.js`: Lógica de negocio + SQL directo.
- `db/init.sql`: Definición de la tabla `tasks`.

Responsabilidad de cada archivo

- `db.js`: Crea el pool de conexiones, un conjunto de conexiones reutilizables a PostgreSQL. No conoce el dominio de tareas.
- `services/tasks.service.js`: Contiene la lógica de negocio y las sentencias SQL. Es el corazón de la aplicación. Cada función recibe el pool como primer parámetro.
- `controllers/tasks.controller.js`: Traduce entre el mundo HTTP y el servicio. Decide los códigos de estado (201, 400, 404, 204). No contiene lógica de negocio propia.
- `routes/tasks.routes.js`: Declara qué combinación de verbo HTTP y ruta invoca a qué función del controlador.
- `app.js`: Fábrica que ensambla la aplicación Express. No levanta ningún servidor. Puede invocarse varias veces con distintos pool, lo que habilita las pruebas de integración.

- `server.js`: Único archivo que efectivamente levanta el servidor HTTP y escucha en un puerto. Es lo que ejecuta el contenedor.
- `db/init.sql`: Script ejecutado automáticamente por PostgreSQL la primera vez que se crea el volumen de datos.

3.3. Flujo de una petición

Cliente HTTP (POST `/tasks` con `{"title": ".Estudiar"}`) → `routes/tasks.routes.js` → detecta POST `/tasks`, invoca `controller.create` → `controllers/tasks.controller.js` llama a `tasksService.createTask(pool, body)` → `services/tasks.service.js` valida el título, ejecuta el INSERT → `db.js` (pool) ejecuta el SQL contra PostgreSQL → Respuesta HTTP 201 con la tarea creada.

4. Preparación del Entorno de Trabajo

4.1. Requisitos previos

- Docker Engine 24 o superior.
- Docker Compose v2 (`docker compose`, sin guion).
- Un editor de texto o IDE (se recomienda Visual Studio Code).
- Cliente HTTP opcional para exploración manual: `curl`, Postman o Insomnia.

4.2. Configuración de credenciales

Las credenciales de la base de datos residen en un único archivo `.env`, que es leído automáticamente por `compose.yml`. Nunca deben escribirse credenciales directamente en el archivo de composición.

```
1 cp .env.example .env
```

Contenido del archivo `.env`:

```
1 POSTGRES_USER=labuser
2 POSTGRES_PASSWORD=labpass
3 POSTGRES_DB=tasksdb
4 POSTGRES_TEST_DB=tasksdb_test
5 PORT=3000
6 DATABASE_URL=postgres://labuser:labpass@localhost:5432/tasksdb
7 TEST_DATABASE_URL=postgres://labuser:labpass@localhost:5433/tasksdb_test
8 BASE_URL=http://localhost:3000
```

Nombre del archivo de composición

La especificación vigente de Compose define `compose.yml` como el nombre canónico del archivo de composición. El nombre heredado `docker-compose.yml` continúa siendo reconocido por compatibilidad, pero no debe emplearse en proyectos nuevos.

4.3. Levantamiento del entorno

```
1 docker compose up -d --build
```

Este comando construye la imagen de la aplicación e inicia tres contenedores:

Servicio	Rol	Puerto local	Persistencia
db	Base de datos de desarrollo y pruebas E2E	5432	Volumen persistente
test-db	Base de datos exclusiva de pruebas de integración	5433	Efímera (<code>tmpfs</code> , en RAM)
app	API REST Node.js + Express	3000	-

Verificación de que la API responde:

```
1 curl http://localhost:3000/health
2 {"status":"ok"}
```

Volumen montado

El archivo `compose.yml` monta la carpeta del proyecto dentro del contenedor (`./usr/src/app`). Esto significa que cualquier archivo de prueba que se agregue o edite en la máquina anfitriona aparece de inmediato dentro del contenedor, sin necesidad de reconstruir la imagen. Un volumen anónimo adicional (`/usr/src/app/node_modules`) preserva las dependencias instaladas durante la construcción de la imagen.

Cuándo sí es necesario reconstruir

Si se modifica `package.json` (por ejemplo, al agregar una nueva dependencia), es obligatorio reconstruir con `docker compose up -d --build`, ya que `node_modules` vive dentro de la imagen y no en la carpeta montada. Si únicamente se agregan o editan archivos `.js`, no se requiere reconstrucción.

4.4. Comandos de ejecución de pruebas

```
1 docker compose exec app npm run test:unit
2 docker compose exec app npm run test:integration
3 docker compose exec app npm run test:e2e
```

También es posible abrir una sesión interactiva dentro del contenedor:

```
1 docker compose exec app sh
```

4.5. Apagado y limpieza

```
1 docker compose down # detiene y elimina los contenedores
2 docker compose down -v # además elimina el volumen de datos de la BD
```

5. Actividad 1: Pruebas Unitarias

5.1. Marco conceptual

Una prueba unitaria verifica una única unidad de código en completo aislamiento. Toda dependencia externa (base de datos, red, sistema de archivos) se sustituye por un doble de prueba o *mock*: un objeto falso que imita el comportamiento de la dependencia real y permite además espiar cómo fue utilizado.

Toda prueba unitaria debe seguir el patrón Arrange-Act-Assert (Preparar, Actuar, Verificar).

5.2. Ejercicio 1.1: Creación exitosa de una tarea

Crear el archivo `tests/unit/tasks.service.test.js`:

```
1 const { createTask } = require("../src/services/tasks.service");
2
3 describe("createTask (prueba unitaria)", () => {
4   test("crea la tarea cuando el titulo es valido", async () => {
5     // 1. PREPARAR (Arrange): construimos un pool falso.
6     // jest.fn() crea una funcion espia que registra sus llamadas.
7     // mockResolvedValue programa la respuesta simulada de la BD.
8     const fakePool = {
9       query: jest.fn().mockResolvedValue({
10         rows: [{ id: 1, title: "Estudiar", done: false }],
11       }),
12     };
13
14     // 2. ACTUAR (Act): invocamos la unidad bajo prueba.
15     const result = await createTask(fakePool, { title: "Estudiar" });
16
17     // 3. VERIFICAR (Assert): comprobamos el resultado y la interaccion.
18     expect(result.id).toBe(1);
19     expect(result.title).toBe("Estudiar");
20     expect(fakePool.query).toHaveBeenCalledTimes(1);
21   });
22 });
```

Error frecuente

Si se omite la palabra reservada `async` antes de `()` en la función del test, Jest arrojará el error `SyntaxError: Unexpected reserved word 'await'`. Regla mnemotécnica: si dentro de una función se usa `await`, esa función debe declararse `async`. Siempre van en pareja.

5.3. Ejercicio 1.2: Validación de título vacío

```
1 test("rechaza la tarea si el titulo esta vacio", async () => {
2   // Pool falso sin respuesta programada: si el servicio lo llamara,
3   // seria un defecto de diseno (no debe tocar la BD con datos
4     invalidos).
5   const fakePool = { query: jest.fn() };
6
7   // Esperamos que la promesa sea RECHAZADA (que lance un error).
8   await expect(
9     createTask(fakePool, { title: "" })
10    ).rejects.toThrow("El titulo es obligatorio");
11 });
```



```

11 // Verificamos que NUNCA se intento consultar la base de datos:
12 expect(fakePool.query).not.toHaveBeenCalled();
13 });

```

5.4. Ejercicio 1.3: Normalización del título (trim)

Verificar que el servicio elimina los espacios en blanco sobrantes antes de persistir el título, inspeccionando los argumentos con los que se invocó la consulta.

```

1  test("elimina los espacios sobrantes del titulo", async () => {
2    const fakePool = {
3      query: jest.fn().mockResolvedValue({
4        rows: [{ id: 1, title: "Estudiar" }],
5      }),
6    };
7
8    await createTask(fakePool, { title: "  Estudiar  " });
9
10   // expect.stringContaining permite validar el SQL sin acoplarse
11   // exactamente.
12   // toHaveBeenCalledWith inspecciona los argumentos reales.
13   expect(fakePool.query).toHaveBeenCalledWith(
14     expect.stringContaining("INSERT"),
15     ["Estudiar"]
16   );
17 });

```

5.5. Ejercicio 1.4: Recurso inexistente devuelve null

```

1  const { getTaskById } = require("../src/services/tasks.service");
2
3  test("getTaskById devuelve null cuando no hay resultados", async () => {
4    {
5      // Simulamos que la BD no devolvio ninguna fila.
6      const fakePool = {
7        query: jest.fn().mockResolvedValue({ rows: [] }),
8      };
9
10     const result = await getTaskById(fakePool, 999);
11     expect(result).toBeNull();
12   }
13 });

```

5.6. Ejercicio 1.5 (propuesto): Rechazo de tipos inválidos

Escribir una prueba que verifique que `createTask` rechaza un título que no sea una cadena de texto, por ejemplo el número 42 o el valor `null`. Debe comprobarse tanto que se lanza el error como que la base de datos nunca fue consultada.

5.7. Ejercicio 1.6 (propuesto): Simulación de un fallo de la base de datos

Utilizando `mockRejectedValue`, simular que PostgreSQL devuelve un error de conexión y verificar que la función `listTasks` propaga dicho error en lugar de silenciarlo.

```

1  const fakePool = {
2    query: jest.fn().mockRejectedValue(new Error("connection refused")),

```

```
3     };
4     // Completar la verificacion...
```

5.8. Resultado esperado

```
1 PASS   tests/unit/tasks.service.test.js
2   createTask (prueba unitaria)
3     [OK] crea la tarea cuando el titulo es valido (8 ms)
4     [OK] rechaza la tarea si el titulo esta vacio (12 ms)
5
6 Test Suites: 1 passed, 1 total
7 Tests:      2 passed, 2 total
8 Snapshots:  0 total
9 Time:       0.792 s
```

6. Actividad 2: Pruebas de Integración

6.1. Marco conceptual

Una prueba de integración verifica que dos o más componentes funcionan correctamente en conjunto. En este caso, se comprueba que la aplicación Express y PostgreSQL colaboran de forma correcta: que las sentencias SQL son válidas, que los tipos de datos coinciden y que las restricciones de la tabla se respetan.

La aplicación se construye en memoria mediante `createApp(pool)` y se interroga con Supertest, que permite emitir peticiones HTTP sin necesidad de abrir un puerto real. La base de datos, en cambio, sí es real: el contenedor `test-db`.

6.2. Andamiaje inicial

Crear el archivo `tests/integration/tasks.api.test.js`:

```
1 const request = require("supertest");
2 const { Pool } = require("pg");
3 const { createApp } = require("../src/app");
4
5 const TEST_DATABASE_URL = process.env.TEST_DATABASE_URL || "postgres://
  labuser:labpass@localhost:5433/tasksdb_test";
6 let pool;
7 let app;
8
9 // Se ejecuta UNA vez, antes de todas las pruebas del archivo.
10 beforeAll(() => {
11   pool = new Pool({ connectionString: TEST_DATABASE_URL });
12   app = createApp(pool);
13 });
14
15 // Se ejecuta UNA vez, al terminar todas las pruebas.
16 afterAll(async () => {
17   await pool.end(); // cierra la conexion para que Jest finalice limpio
18 });
19
20 // Se ejecuta ANTES DE CADA prueba individual.
21 beforeEach(async () => {
22   // Cada prueba debe ser independiente de las demas.
23   await pool.query("TRUNCATE TABLE tasks RESTART IDENTITY");
24 });
```

6.3. Ejercicio 2.1: Persistencia efectiva de una tarea

```
1 describe("POST /tasks (integracion con PostgreSQL real)", () => {
2   test("crea una tarea y queda guardada en la base de datos", async ()
3     => {
4     // 1. ACTUAR: peticion HTTP contra la app en memoria.
5     const res = await request(app)
6       .post("/tasks")
7       .send({ title: "Estudiar para el examen" });
8
9     // 2. VERIFICAR LA RESPUESTA HTTP.
10    expect(res.status).toBe(201);
11    expect(res.body.title).toBe("Estudiar para el examen");
12
13    // 3. VERIFICAR CONTRA LA BASE DE DATOS REAL.
```

```

13 // Esto es lo que distingue a una prueba de integracion.
14 const { rows } = await pool.query("SELECT * FROM tasks WHERE id = $1
    ", [res.body.id]);
15 expect(rows).toHaveLength(1);
16 expect(rows[0].title).toBe("Estudiar para el examen");
17 });
18 });

```

6.4. Ejercicio 2.2: La validación impide la escritura

```

1 test("responde 400 si falta el titulo y no inserta nada", async () =>
  {
2     const res = await request(app).post("/tasks").send({});
3     expect(res.status).toBe(400);
4
5     // Confirmamos contra la BD real que NO se inserto ninguna fila.
6     const { rows } = await pool.query("SELECT * FROM tasks");
7     expect(rows).toHaveLength(0);
8 });

```

6.5. Ejercicio 2.3: Recurso inexistente devuelve 404

```

1 test("GET /tasks/:id devuelve 404 si la tarea no existe", async () =>
  {
2     const res = await request(app).get("/tasks/9999");
3     expect(res.status).toBe(404);
4 });

```

6.6. Ejercicio 2.4: Actualización del estado done

```

1 test("PUT /tasks/:id actualiza el estado en la base de datos", async
  () => {
2     // Primero creamos la tarea que vamos a actualizar.
3     const creada = await request(app)
4       .post("/tasks")
5       .send({ title: "Lavar el auto" });
6
7     const res = await request(app)
8       .put(`/tasks/${creada.body.id}`)
9       .send({ done: true });
10
11     expect(res.status).toBe(200);
12     expect(res.body.done).toBe(true);
13
14     const { rows } = await pool.query("SELECT done FROM tasks WHERE id =
        $1", [creada.body.id]);
15     expect(rows[0].done).toBe(true);
16 });

```

6.7. Ejercicio 2.5: Eliminación efectiva de la fila

```

1  test("DELETE /tasks/:id elimina la fila de la base de datos", async ()
    => {
2      const creada = await request(app)
3          .post("/tasks")
4          .send({ title: "Tarea temporal" });
5
6      const res = await request(app).delete(`/tasks/${creada.body.id}`);
7      expect(res.status).toBe(204);
8
9      const { rows } = await pool.query("SELECT * FROM tasks WHERE id = $1
        ", [creada.body.id]);
10     expect(rows).toHaveLength(0);
11 });

```

6.8. Ejercicio 2.6 (propuesto): Independencia entre pruebas

Comentar temporalmente el bloque `beforeEach` que ejecuta el `TRUNCATE` y volver a correr la suite completa. Documentar qué prueba falla, por qué falla y qué principio de diseño de pruebas se está violando.

6.9. Ejercicio 2.7 (propuesto): Verificación del listado

Escribir una prueba que inserte tres tareas mediante `POST /tasks`, luego solicite `GET /tasks` y verifique que se devuelven exactamente tres elementos, ordenados por ID ascendente.

6.10. Resultado esperado

```

1 PASS  tests/integration/tasks.api.test.js
2  POST /tasks (integracion con Postgres real)
3    [OK] crea una tarea y queda guardada en la base de datos (108 ms)
4    [OK] responde 400 si falta el titulo (10 ms)
5
6 Test Suites: 1 passed, 1 total
7 Tests:      2 passed, 2 total
8 Time:       1.433 s

```

Observación sobre los tiempos

El primer caso tarda 108 ms y el segundo apenas 10 ms. La diferencia corresponde al costo fijo de abrir la conexión con PostgreSQL, que se amortiza en las pruebas siguientes.

7. Actividad 3: Pruebas End-to-End

7.1. Marco conceptual

Una prueba end-to-end ejercita el sistema completo en ejecución, de extremo a extremo, desde la perspectiva de un cliente real. Es una prueba de caja negra: el archivo de prueba no importa absolutamente nada del código fuente. Solo emite peticiones HTTP reales, exactamente igual que lo haría un navegador, una aplicación móvil o Postman.

Requisito previo

El stack completo debe estar en ejecución: `docker compose up -d`. Si el servicio `app` no está levantado, todas las peticiones fallarán con `ECONNREFUSED`.

7.2. Ejercicio 3.1: Verificación de disponibilidad

Crear el archivo `tests/e2e/tasks.e2e.test.js`:

```
1 // Dentro del contenedor "app", BASE_URL ya viene definida por Compose.
2 // El valor de respaldo sirve para ejecucion local.
3 const BASE_URL = process.env.BASE_URL || "http://localhost:3000";
4
5 test("el servicio responde en /health", async () => {
6   const res = await fetch(`${BASE_URL}/health`);
7   const body = await res.json();
8
9   expect(res.status).toBe(200);
10  expect(body.status).toBe("ok");
11 });
```

7.3. Ejercicio 3.2: Flujo completo del ciclo de vida de una tarea

```
1 describe("Flujo completo de una tarea (E2E)", () => {
2   test("crear -> consultar -> completar -> eliminar", async () => {
3     // PASO 1: CREAR (fetch() emite una peticion HTTP REAL por la red)
4     const resCrear = await fetch(`${BASE_URL}/tasks`, {
5       method: "POST",
6       headers: { "Content-Type": "application/json" },
7       body: JSON.stringify({ title: "Preparar la practica" }),
8     });
9     expect(resCrear.status).toBe(201);
10    const tarea = await resCrear.json();
11    const id = tarea.id;
12
13    // PASO 2: CONSULTAR
14    const resConsultar = await fetch(`${BASE_URL}/tasks/${id}`);
15    expect(resConsultar.status).toBe(200);
16    const consultada = await resConsultar.json();
17    expect(consultada.done).toBe(false);
18
19    // PASO 3: COMPLETAR
20    const resCompletar = await fetch(`${BASE_URL}/tasks/${id}`, {
21      method: "PUT",
22      headers: { "Content-Type": "application/json" },
23      body: JSON.stringify({ done: true }),
24    });
25    expect(resCompletar.status).toBe(200);
```

```

26     const completada = await resCompletar.json();
27     expect(completada.done).toBe(true);
28
29     // PASO 4: ELIMINAR
30     const resEliminar = await fetch(`${BASE_URL}/tasks/${id}`, {
31       method: "DELETE",
32     });
33     expect(resEliminar.status).toBe(204);
34
35     // PASO 5: CONFIRMAR QUE YA NO EXISTE
36     const resFinal = await fetch(`${BASE_URL}/tasks/${id}`);
37     expect(resFinal.status).toBe(404);
38   });
39 });

```

7.4. Ejercicio 3.3 (propuesto): Rechazo de datos inválidos de extremo a extremo

Escribir una prueba E2E que envíe un POST `/tasks` con cuerpo vacío y verifique que el sistema completo responde con código 400. Reflexionar: esta prueba, ¿aporta información que la prueba unitaria equivalente no aportara ya?

7.5. Ejercicio 3.4 (propuesto): Idempotencia del borrado

Escribir una prueba E2E que cree una tarea, la elimine dos veces consecutivas y verifique que la primera eliminación responde 204 y la segunda 404.

7.6. Ejercicio 3.5 (propuesto): Sondeo del listado

Escribir una prueba E2E que consulte GET `/tasks`, verifique que la respuesta es un arreglo y que el encabezado Content-Type contiene `application/json`.

7.7. Resultado esperado

```

1 PASS   tests/e2e/tasks.e2e.test.js
2     Flujo completo de una tarea (E2E)
3       [OK] crear -> consultar -> completar -> eliminar (208 ms)
4
5 Test Suites: 1 passed, 1 total
6 Tests:      1 passed, 1 total
7 Time:       0.953 s

```

Sobre el flag `--runInBand`

Los scripts de integración y E2E incorporan el flag `--runInBand`, que obliga a Jest a ejecutar las suites en secuencia y no en paralelo. Es indispensable porque ambas comparten una misma base de datos: dos pruebas ejecutándose simultáneamente podrían sobrescribirse mutuamente los datos. Las pruebas unitarias, al usar mocks aislados, sí pueden paralelizarse sin riesgo.

8. Actividad 4: Evolución del Modelo de Datos

8.1. Enunciado

Agregar al dominio un nuevo atributo `priority` (prioridad), de tipo texto, con valores admitidos `alta`, `media` y `baja`, y valor por defecto `media`. El estudiante deberá propagar el cambio a través de todas las capas del sistema y de los tres niveles de prueba.

8.2. Paso 1: Esquema de la base de datos

Modificar `db/init.sql`:

```
1 CREATE TABLE IF NOT EXISTS tasks (  
2   id SERIAL PRIMARY KEY,  
3   title VARCHAR(255) NOT NULL,  
4   done BOOLEAN NOT NULL DEFAULT false,  
5   priority VARCHAR(20) NOT NULL DEFAULT 'media',  
6   created_at TIMESTAMP NOT NULL DEFAULT NOW()  
7 );
```

Atención

El script `init.sql` solo se ejecuta la primera vez que PostgreSQL crea su volumen de datos. Para que el cambio surta efecto es obligatorio destruir el volumen y recrear el entorno:

```
1 docker compose down -v && docker compose up -d --build
```

8.3. Paso 2: Capa de servicio

Modificar `src/services/tasks.service.js`:

```
1 async function createTask(pool, { title, priority }) {  
2   if (!title || typeof title !== "string" || !title.trim()) {  
3     const err = new Error("El titulo es obligatorio");  
4     err.status = 400;  
5     throw err;  
6   }  
7  
8   const { rows } = await pool.query(  
9     "INSERT INTO tasks (title, done, priority) " +  
10    "VALUES ($1, false, $2) " +  
11    "RETURNING id, title, done, priority, created_at",  
12    [title.trim(), priority || "media"]  
13  );  
14  
15  return rows[0];  
16 }
```

Adicionalmente, agregar la columna `priority` a las sentencias `SELECT` de `listTasks` y `getTaskById`.

8.4. Paso 3: Actualización de los tres niveles de prueba

Nivel unitario

```
1 test("crea la tarea con la prioridad indicada", async () => {  
2   const fakePool = {  
3     query: jest.fn().mockResolvedValue({
```



```

4       rows: [{ id: 1, title: "Estudiar", priority: "alta" }],
5     }),
6   });
7
8   const result = await createTask(fakePool, { title: "Estudiar",
9     priority: "alta" });
10  expect(result.priority).toBe("alta");
11  });

```

Nivel de integración

```

1  test("guarda la prioridad en la base de datos", async () => {
2    const res = await request(app)
3      .post("/tasks")
4      .send({ title: "Urgente", priority: "alta" });
5
6    expect(res.body.priority).toBe("alta");
7
8    const { rows } = await pool.query("SELECT priority FROM tasks WHERE
9      id = $1", [res.body.id]);
10   expect(rows[0].priority).toBe("alta");
11 });

```

Nivel end-to-end

```

1  const resCrear = await fetch(`${BASE_URL}/tasks`, {
2    method: "POST",
3    headers: { "Content-Type": "application/json" },
4    body: JSON.stringify({
5      title: "Preparar la practica",
6      priority: "alta",
7    }),
8  });
9  const tarea = await resCrear.json();
10 expect(tarea.priority).toBe("alta");

```

8.5. Resultado esperado y diagnóstico de errores

Los tres niveles deben permanecer en verde. Si alguno falla, el mensaje indica exactamente qué capa quedó desactualizada:

Síntoma	Causa probable
column "priority"does not exist	No se recreó el volumen: falta docker compose down -v
priority llega como undefined. La prueba unitaria pasa pero la de integración falla.	Se omitió la columna en el SELECT o en la cláusula RETURNING . El mock devuelve un dato que la base de datos real no produce.

Reflexión de ingeniería

Un único atributo nuevo obliga a modificar cuatro capas y tres suites de prueba. Esta observación permite dimensionar el costo real de la evolución del software y justifica la importancia de una arquitectura desacoplada.

9. Actividad 5: Inyección Deliberada de un Defecto

9.1. Enunciado

Se introducirá intencionalmente un defecto en el código de producción y se observará en qué nivel de prueba se detecta primero, con qué costo y con qué claridad diagnóstica.

9.2. Paso 1: Introducción del defecto

En `src/services/tasks.service.js`, comentar el bloque de validación:

```
1 async function createTask(pool, { title } = {}) {
2   // VALIDACION COMENTADA A PROPOSITO
3   /*
4   if (!title || typeof title !== "string" || !title.trim()) {
5     const err = new Error("El titulo es obligatorio");
6     err.status = 400;
7     throw err;
8   }
9   */
10
11   const { rows } = await pool.query(
12     "INSERT INTO tasks (title, done) " +
13     "VALUES ($1, false) " +
14     "RETURNING id, title, done, created_at",
15     [title]
16   );
17   return rows[0];
18 }
```

Observación metodológica

Los archivos de prueba no se modifican. Se altera únicamente el código de producción. Ese es precisamente el propósito de una suite de pruebas: actuar como red de seguridad que advierte cuando el comportamiento del sistema cambia sin autorización.

9.3. Paso 2: Ejecución en orden ascendente de la pirámide

Nivel unitario

```
1 docker compose exec app npm run test:unit
```

Salida esperada:

```
1 FAIL   tests/unit/tasks.service.test.js
2   createTask (prueba unitaria)
3     [OK] crea la tarea cuando el titulo es valido (7 ms)
4     [X] rechaza la tarea si el titulo esta vacio (14 ms)
5
6   createTask > rechaza la tarea si el titulo esta vacio
7     expect(received).rejects.toThrow()
8     Received promise resolved instead of rejected
9     Resolved to value: undefined
10
11 Test Suites: 1 failed, 1 total
12 Tests:      1 failed, 1 passed, 2 total
13 Time:       0.79 s
```

Interpretación: El mensaje `Received promise resolved instead of rejected` significa: "la prueba esperaba que la función lanzara un error, pero la función terminó correctamente". El defecto fue detectado en menos de un segundo, sin haber levantado PostgreSQL ni emitido una sola petición HTTP.

Nivel de integración

```
1 docker compose exec app npm run test:integration
```

El defecto también se manifiesta, pero de forma más tardía y menos precisa: PostgreSQL rechaza el `INSERT` por la restricción `NOT NULL` de la columna `title`, y la aplicación responde con un código 500 genérico en lugar del 400 esperado.

```
1 expect(res.status).toBe(400)
2 Expected: 400
3 Received: 500
```

Nivel end-to-end

Si el flujo E2E únicamente ejercita el camino feliz (crear una tarea con título válido), la suite permanecerá en verde a pesar de que el defecto existe.

9.4. Conclusión de la actividad

Hallazgo central: El defecto fue detectado primero, más rápido y con mayor claridad diagnóstica por la prueba unitaria. La prueba de integración lo detectó de forma tardía y ambigua. La prueba E2E no lo detectó en absoluto, porque no lo verificaba explícitamente.

Corolario: una suite E2E en verde no constituye evidencia de ausencia de defectos. La cobertura de casos negativos pertenece principalmente al nivel unitario.

9.5. Paso 3: Restauración

Descomentar el bloque de validación, o bien revertir el archivo mediante control de versiones:

```
1 git checkout src/services/tasks.service.js
```

10. Actividad 6: Medición Empírica de la Pirámide de Pruebas

10.1. Enunciado

Cronometrar la ejecución de cada nivel de prueba y sustentar cuantitativamente la distribución recomendada por la pirámide de pruebas.

10.2. Procedimiento

```
1 docker compose exec app sh -c "time npm run test:unit"
2 docker compose exec app sh -c "time npm run test:integration"
3 docker compose exec app sh -c "time npm run test:e2e"
```

Nota técnica

Se emplea `sh -c "time..."` porque `time` es una construcción interna del intérprete de comandos y no un binario independiente. Invocarlo directamente como `docker compose exec app time npm run` producirá un error de comando no encontrado.

10.3. Tabla de registro

El estudiante debe completar la siguiente tabla con sus mediciones:

Nivel	N. de pruebas	Tiempo real	Recursos que ejercita
Unitario			Solo memoria; dobles de prueba.
Integración			PostgreSQL real (<code>test-db</code>).
End-to-End			Servidor HTTP real + PostgreSQL real.

10.4. Análisis solicitado

El estudiante deberá responder de forma argumentada:

1. ¿Por qué el primer caso de la suite de integración tarda significativamente más que los siguientes?
2. Extrapole los tiempos medidos a un escenario hipotético de 1000 pruebas por nivel. ¿Cuánto tardaría cada suite? ¿Qué implicancias tiene esto sobre un pipeline de integración continua que se ejecuta en cada commit?
3. ¿Qué proporción de pruebas por nivel recomendaría usted para un proyecto real? Justifique con los datos obtenidos.
4. Una fracción del tiempo medido corresponde al arranque del proceso de Node.js y de Jest, no a la ejecución de las pruebas. Proponga un procedimiento de medición que minimice este sesgo.

11. Indicaciones Generales del Caso

11.1. Modalidad de trabajo

- El trabajo se desarrollará en equipos de dos estudiantes (parejas de programación), rotando el rol de conductor y navegante en cada actividad.
- Cada equipo trabajará sobre su propio repositorio Git, con commits atómicos y mensajes descriptivos. Se exige un commit independiente por cada actividad completada.

11.2. Condiciones de ejecución

- Todas las pruebas deben ejecutarse dentro del contenedor, mediante `docker compose exec app`. No se aceptarán evidencias de ejecución en el entorno local del anfitrión.
- El archivo `.env` no debe versionarse. Únicamente se versiona `.env.example`. Verifique que `.gitignore` lo contemple.
- Ninguna credencial debe escribirse directamente en `compose.yml`.
- Cada prueba debe ser independiente y repetible: su resultado no puede depender del orden de ejecución ni del estado dejado por otra prueba.

11.3. Criterios de calidad del código de prueba

- Toda prueba debe seguir explícitamente el patrón Arrange-Act-Assert, delimitado con comentarios.
- El nombre de cada prueba debe describir el comportamiento esperado en lenguaje natural, no el nombre del método invocado.
- Una prueba unitaria no debe abrir conexiones de red ni acceder a la base de datos. Si lo hace, es una prueba de integración mal clasificada.
- Las aserciones deben ser específicas: `expect(res.status).toBe(400)` es preferible a `expect(res.status)`.

11.4. Errores comunes y su prevención

Error	Prevención
<code>SyntaxError: Unexpected reserved word 'await'</code>	Toda función que use <code>await</code> debe declararse <code>async</code> .
Jest no finaliza y queda colgado	Falta cerrar el pool en <code>afterAll</code> con <code>await pool.end()</code> .
<code>ECONNREFUSED</code> en las pruebas E2E	El stack no está levantado: <code>docker compose up -d</code> .
Pruebas que pasan aisladas pero fallan en conjunto	Falta el <code>TRUNCATE</code> en <code>beforeEach</code> , o se omitió <code>--runInBand</code> .
Una dependencia nueva no se encuentra	Se modificó <code>package.json</code> sin reconstruir: <code>docker compose up -d --build</code> .

12. Entregables de la Evaluación

Cada equipo deberá presentar los siguientes productos:

1. **Repositorio Git:** Debe contener el proyecto completo con las tres carpetas de prueba pobladas (`tests/unit`, `tests/integration`, `tests/e2e`), incluidos los ejercicios propuestos, y un historial de commits que evidencie el avance por actividad.
2. **Informe Técnico de Pruebas:** La estructura y organización interna del documento quedan a criterio del equipo, a fin de fomentar la propuesta analítica propia. Sin embargo, se espera un reporte profesional que contenga como mínimo:
 - Evidencia (capturas) de la ejecución de los tres niveles de prueba dentro del contenedor.
 - La tabla de tiempos de la Actividad 6, debidamente completada.
 - El análisis argumentado solicitado en la Actividad 6.
 - Las conclusiones extraídas de la Actividad 5 respecto de la capacidad diagnóstica de cada nivel.
 - Una propuesta de métricas cuantitativas de calidad para el proyecto, con sus respectivas ecuaciones de obtención (por ejemplo, cobertura de líneas, cobertura de ramas, densidad de defectos, razón de pruebas por nivel).
3. **Presentación Ejecutiva:** Exposición profesional en diapositivas para sustentar los hallazgos, la arquitectura del entorno de pruebas y las conclusiones sobre la pirámide de pruebas frente a la clase. La estructura y el enfoque serán definidos libremente por el equipo.